

Chapter 58. Intelligent Automation

The purchase-order pipeline that writes: idempotency, transactions, audit trails, approval thresholds, and ROI counted honestly.

Draft chapter

Contents

Chapter 58. Intelligent Automation

Why this matters

In plain English

58.1 Three generations of automation

58.2 What to automate, and what to leave alone

58.3 The pipeline, assembled

58.4 Writing to a system of record

58.5 The human in the loop

58.6 Exceptions, classified

58.7 Measuring it honestly

58.8 Rolling it out

58.9 The people part

58.10 When not to automate

Common mistakes and how to spot them

In the real world: the week the pilot was nearly cancelled

Tools

The project: the intake pipeline, end to end

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 58. Intelligent Automation

Part VI — Production ML, Generative AI & MLOps

Chapter at a glance

You will learn to: tell the three generations of automation apart and pick the right one · choose what to automate using volume, variability, rules, and the cost of an error · assemble Chapters 54 to 57 into a pipeline that writes to a system of record · make writes **idempotent**, transactional, and auditable · design an approval threshold priced in money rather than guessed · classify exceptions so each one gets retried, held, escalated, or rejected · measure straight-through rate *and* silent error rate, which are the same system described two ways · do the ROI arithmetic honestly, including the review time and the cost of being wrong · roll out in stages with a gate at each one · and handle the conversation with the person whose job you just changed.

Before you start: Chapter 54 (extraction, validation), Chapter 55 (tools that act), Chapter 57 (tolerant parsing, metering, fallbacks), Chapter 12 (SQL and transactions), Chapter 29 (retries, idempotency).

Time needed: 14–18 hours, spread over three weeks.

Tools: Python 3.12, SQLite (the standard library's `sqlite3`), and the companion pipeline from Chapters 54 and 57.

Practice system: Riverstone's purchase-order intake. The 60 emails from Chapter 54, a real SQLite order book with an audit log, and the approval rules that decide what reaches it.

Why this matters

This is the chapter Chapter 1 promised. The book opened with Riverstone's sales desk retyping purchase orders out of emails, and every part since has added a piece: SQL to store them, Python to read them, a model to extract them, evaluation to trust the extraction, and operations to keep it running.

Now the last step, which is different in kind from all the others: **the pipeline writes**. Until now a wrong answer was a sentence on a screen. From here, a wrong answer is a row in the order book, a picking list in the warehouse, and a lorry going to the wrong address.

That change brings four questions that have nothing to do with models:

- **How do I make sure the same email never creates two orders?**
- **What gets written automatically, and what waits for a person?**
- **When it goes wrong, who is accountable, and what does the log say?**
- **Is this actually worth doing?** The honest arithmetic in section 58.7 says something this chapter's own pipeline does not want to hear.

In plain English

Automation is not replacing a person with a program. It is moving a person from typing to deciding.

Riverstone's sales coordinator reads an email, works out what was ordered, types it into the ERP, and moves on. Three of those four steps are mechanical and one is judgment. A good automation does the mechanical three and hands the judgment back, with the evidence attached.

That framing decides everything else in the chapter:

- **The queue is the product.** A pipeline that processes 88% of emails and hands over 12% with clear reasons is worth far more than one that processes 100% and is quietly wrong on a fifth of them.
- **The write is the dangerous part.** Reading is reversible, writing is not, so the write needs a uniqueness rule, a transaction, and a log.
- **The threshold is a business decision.** "Which orders can go straight through?" is a question about money, and it has an answer you can compute.
- **The person is not the obstacle.** They are the escalation path, the quality check, and the only one who knows when the model's confident answer is nonsense.

58.1 Three generations of automation

Generation	How it works	Good at	Breaks when
Macros and RPA	a robot clicks through the user interface a human would use	legacy systems with no API, where change is impossible	the screen moves one pixel, or the system is upgraded
API and integration	systems talk directly, with contracts	anything with an interface; the durable answer	the data is unstructured, so there is nothing to send
Model-assisted	a model turns unstructured input into structured data, then integration takes over	email, documents, photographs, free text	the model is confidently wrong and nothing checks it

Riverstone's purchase orders need the third feeding the second: a model reads the email, and an ordinary insert writes the order. **The model is the smallest part of the system and the only part anyone wants to talk about.**

Watch out: RPA is a bridge, not a destination. Screen-scraping robots are sold as "no integration needed" and their maintenance cost is real, ongoing, and invisible at purchase. They earn their place where a system truly has no API and cannot be changed, and every such deployment should carry an expiry date and a plan to replace it.

58.2 What to automate, and what to leave alone

Four questions, in this order:

Question	Automate when	Leave alone when
Volume	it happens many times a day	it happens twice a month
Variability	inputs vary within a describable range	every case really is different
Rules	the decision can be written down	the decision needs context nobody has captured
Cost of an error	a wrong result is caught quickly and cheaply	a wrong result is expensive and silent

Riverstone's purchase orders score well on the first three and badly on the fourth, which is exactly why this chapter ends where it does. Twenty to forty emails a day, five recognizable shapes, rules that finance already wrote, and an error that becomes a wrong delivery two days later.

The trap is to rank candidates by annoyance. The most-complained-about task is often low volume and high judgment, which makes it the worst candidate and the most requested one. Rank by **volume × time per item**, then filter by the cost of an error.

58.3 The pipeline, assembled

```
import sys
sys.path.insert(0, ".")
import erp
from intake import run, APPROVAL_LIMIT
from pipeline import load_emails
from provider import Meter

erp.rebuild()
truth, emails = load_emails()
meter = Meter()
result = run(emails, truth, meter=meter)

total = sum(result["counts"].values())
print(f"{total} emails processed at 06:00")
for outcome, count in sorted(result["counts"].items()):
    print(f"  {outcome.replace('_', ' '):<20} {count:>3}")
print(f"\nstraight-through rate: {result['counts']['loaded'] / total:.0%}")
print(f"model cost for the run: Rs {meter.cost_rupees():.2f}")
print(f"approval limit in force: Rs {APPROVAL_LIMIT:,}")
```

```
60 emails processed at 06:00
  awaiting approval      6
  held for review       1
  loaded                53

straight-through rate: 88%
model cost for the run: Rs 4.74
approval limit in force: Rs 100,000
```

Riverstone's purchase-order intake, end to end

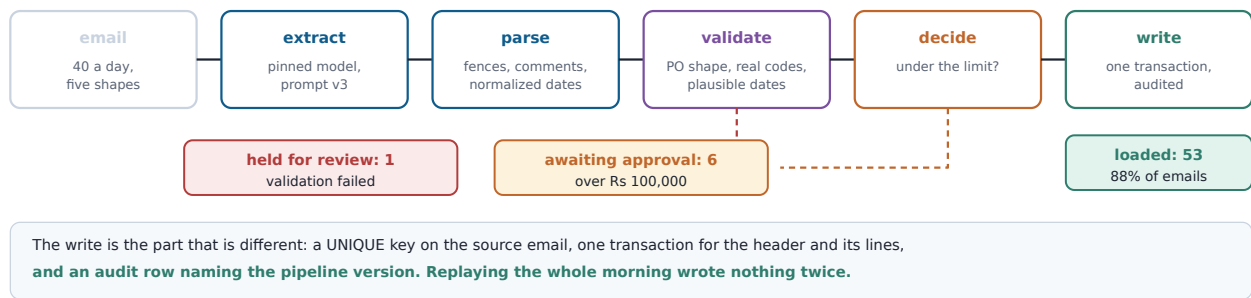


Figure 58.1 – The model is the second box. Everything that makes this safe is in the last one.

Line by line: `erp.rebuild()` starts from an empty order book so the chapter is reproducible. `run` does the whole morning: for each email it calls the model with Chapter 57's prompt v3, parses tolerantly, validates against business rules, and then either writes the order, writes it as *awaiting approval*, or holds it for a person. The counts are what a dashboard would show.

The pipeline's stages, and where each came from:

Stage	What it does	From
Read	fetch the email	this chapter (a folder stands in for the mailbox)
Extract	model call, pinned version, prompt v3	Chapters 54 and 57
Parse	fences, comments, normalized dates	Chapter 57
Validate	PO shape, real product codes, plausible dates and quantities	Chapter 54
Decide	load, hold for approval, or queue for a person	this chapter
Write	one transaction, idempotent, audited	this chapter
Report	counts, queue with reasons, cost	Chapters 56 and 57

58.4 Writing to a system of record

Three properties, and the code that gives them.

Idempotency: one email, one order

```
CREATE TABLE orders (
  order_id      INTEGER PRIMARY KEY AUTOINCREMENT,
  po_number     TEXT      NOT NULL,
  customer      TEXT      NOT NULL,
  delivery_date TEXT      NOT NULL,
  order_value   REAL      NOT NULL,
  source_email  TEXT      NOT NULL UNIQUE,    -- the idempotency key: one email, one order
  status        TEXT      NOT NULL,          -- loaded | awaiting_approval
  created_at    TEXT      NOT NULL
);
```

Line by line: `source_email ... UNIQUE` is the whole mechanism. The database, not the application, enforces that one email produces at most one order, so a retry, a resend, a duplicate run at 06:05, and two workers racing all end in the same state. **Put the guarantee in the database**, because application-level checks lose races and get refactored away.

Prove it by replaying the entire morning:

```
erp.rebuild()
run(emails, truth)                                # the 06:00 run
before = erp.connect().execute("SELECT COUNT(*) AS n FROM orders").fetchone()["n"]
replay = run(emails, truth)                        # the same emails again, by accident
connection = erp.connect()
after = connection.execute("SELECT COUNT(*) AS n FROM orders").fetchone()["n"]

print(f"orders before the replay: {before}")
print(f"orders after replaying every email: {after}")
print(f"duplicates ignored on the replay: {replay['counts'].get('duplicate_ignored', 0)}")
print(f"audit entries recording the duplicates: "
      f"{connection.execute('SELECT COUNT(*) AS n FROM audit_log WHERE action = %s' %
                             repr('duplicate_ignored')).fetchone()['n']}")
```

```
orders before the replay: 59
orders after replaying every email: 59
duplicates ignored on the replay: 53
audit entries recording the duplicates: 59
```

Nothing was written twice, and **the attempts were still recorded**. That distinction matters in an audit: "we ignored 59 duplicate submissions" is a fact somebody will eventually need.

A transaction: the header and the lines together

```
with connection:                                # one transaction: header and lines together, or neither
    cursor = connection.execute("INSERT INTO orders (...) VALUES (...)", (...))
    order_id = cursor.lastrowid
    for item in order["items"]:
        connection.execute("INSERT INTO order_lines (...) VALUES (...)", (...))
```

Line by line: `with connection:` commits at the end of the block and rolls back if anything raises. Without it, a failure between the header insert and the third line insert leaves an order with two of its three lines, which is the worst possible outcome: it looks complete and ships short. **Any write that spans more than one table belongs in a transaction** (Chapter 12).

An audit log: what was proposed, what was written, by what

```
erp.rebuild()
run(emails, truth)
run(emails, truth)                                # two runs, so the duplicates appear
connection = erp.connect()
print("what the audit log holds after two runs:")
for row in connection.execute("SELECT action, COUNT(*) AS n FROM audit_log GROUP BY action ORDER BY
n DESC"):
    print(f"  {row['action']:<20} {row['n']:>4}")

print("\none email's history:")
for row in connection.execute(
    "SELECT at, action, actor, detail FROM audit_log WHERE source_email = 'email_004' ORDER BY
entry_id"):
    print(f"  {row['at']}  {row['action']:<18} {row['actor']}  {row['detail']}")
```

what the audit log holds after two runs:

proposed	120
duplicate_ignored	59
loaded	53
awaiting_approval	6

one email's history:

2026-03-02T06:00:00	proposed	intake-v1 (prompt v3, model v1)	clean
2026-03-02T06:00:00	loaded	intake-v1 (prompt v3, model v1)	order 4, 2 lines
2026-03-02T06:00:00	proposed	intake-v1 (prompt v3, model v1)	clean
2026-03-02T06:00:00	duplicate_ignored	intake-v1 (prompt v3, model v1)	already order 4

What the log has to contain, in any system that writes automatically:

Field	The question it answers
Source identifier	which input caused this?
Action	proposed, loaded, held, rejected, approved
Actor	which pipeline version, or which person
Detail	the reason, in words a person can read
Timestamp	when, to the second

"The model proposed it" is not an accountable answer. The pipeline version is the accountable answer, and the approval record names the human when there was one. Chapter 64 covers the governance; this is the minimum that makes it possible.

58.5 The human in the loop

Two gates, and they do different jobs.

- **Validation** (Chapter 54) catches malformed output: no PO number, an unknown product code, a date in 1926. Riverstone holds those for a person and writes nothing.
- **The approval threshold** catches *large* output. A correctly extracted order for ₹8 lakh is well-formed and worth a human glance, because the cost of being wrong scales with the value.

```

for limit in (0, 50_000, 100_000, 200_000):
    erp.rebuild()
    outcome = run(emails, truth, approval_limit=limit)["counts"]
    loaded = outcome.get("loaded", 0)
    approvals = outcome.get("awaiting_approval", 0)
    held = outcome.get("held_for_review", 0)
    print(f"limit Rs {limit:>7,}: loaded {loaded:>2}, awaiting approval {approvals:>2}, "
          f"held for review {held}, straight through {loaded / 60:.0%}")

```

```

limit Rs      0: loaded  0, awaiting approval 59, held for review 1, straight through 0%
limit Rs  50,000: loaded 40, awaiting approval 19, held for review 1, straight through 67%
limit Rs 100,000: loaded 53, awaiting approval  6, held for review 1, straight through 88%
limit Rs 200,000: loaded 59, awaiting approval  0, held for review 1, straight through 98%

```

Line by line: the limit is the only thing changing. At ₹0 everything waits for a person, which is not automation at all; with no limit everything is written unseen. The rows in between are the actual choice.

Designing the queue, which is where most automation projects are won or lost:

- **One screen**, showing the email beside the proposed order. The reviewer's job is to compare, not to retype.
- **The reason, in words:** "delivery date not YYYY-MM-DD: '04 Mar'", not "validation error 7".
- **Accept, correct, or reject**, with correction as the common case. A corrected order is also a training example and a golden-set candidate (Chapter 57).
- **Bounded:** if the queue is longer than a person can clear in a morning, the thresholds are wrong, not the person.
- **Measured:** queue length, time to clear, and correction rate per reason. The reason that generates the most corrections is your next engineering task.

58.6 Exceptions, classified

Every automation needs a taxonomy, because "it failed" is not an instruction.

Class	Example	What the pipeline does
Transient	the provider rate-limits; the ERP is restarting	retry with backoff (Chapter 57), then queue
Malformed input	the email is a PDF photo, or an out-of-office reply	route to a different handler, or reject with a reason
Uncertain extraction	validation fails; two POs in one email	hold for a person, with the email attached
Policy	order value over the limit; a customer on credit hold	write as pending, notify the owner
Unknown reference	a product code that isn't ours	reject to a person: never guess a foreign key
Duplicate	the same email arrives twice	ignore, and record that you did

Class	Example	What the pipeline does
Systemic	60 of 60 unparseable (Chapter 57's provider change)	stop the run, alert, process nothing

That last row deserves its own rule: **a pipeline should refuse to continue when the failure rate crosses a threshold**. Processing 60 emails wrongly is much worse than processing none, and a circuit breaker on the *batch* is one line of code.

58.7 Measuring it honestly

Here is the number everybody quotes, and the number that decides whether this should ship.

```

erp.rebuild()
run(emails, truth)
connection = erp.connect()

correct = wrong = 0
for row in connection.execute("SELECT order_id, source_email, po_number, customer, delivery_date "
                              "FROM orders WHERE status = 'loaded'"):
    want = truth[row["source_email"]]
    lines = [(line["product_code"], line["quantity"]) for line in
              connection.execute("SELECT product_code, quantity FROM order_lines WHERE order_id =
?",
                                (row["order_id"],))]
    matches = (row["po_number"] == want["po_number"] and row["customer"] == want["customer"]
               and row["delivery_date"] == want["delivery_date"]
               and sorted(lines) == sorted((i["product_code"], i["quantity"]) for i in
                                           want["items"]))
    correct += matches
    wrong += not matches

loaded = correct + wrong
print(f"orders loaded without a human: {loaded}")
print(f"  of those, correct:          {correct}")
print(f"  of those, silently wrong:   {wrong}   ({wrong / loaded:.0%}")
print(f"\nstraight-through rate: {loaded / 60:.0%}   silent error rate: {wrong / loaded:.0%}")

```

```

orders loaded without a human: 53
  of those, correct:          43
  of those, silently wrong:   10   (19%)

straight-through rate: 88%   silent error rate: 19%

```

The same system, described two ways



Straight through saves the most labour and costs the most money. The recommendation this chapter's own numbers force is the **middle row: the model drafts every order, a person confirms it in 45 seconds, and nothing reaches the ERP unseen.**

Figure 58.2 – The second panel is the one missing from most business cases.

Those two percentages describe the same system, and only one of them appears in most business cases. An 88% straight-through rate sounds like a success. A 19% silent error rate means roughly one order in five that nobody looks at is wrong: a wrong quantity, a wrong date, a missing line.

Now price it.

```
EMAILS_PER_DAY = 40
MINUTES_MANUAL = 3.0          # read the email, type it into the ERP
MINUTES_REVIEW = 0.75        # compare a proposed order with the email and accept it
MINUTES_QUEUE = 2.0          # handle one exception properly
COST_PER_HOUR = 300          # a coordinator's fully loaded cost, rupees
COST_OF_A_WRONG_ORDER = 2_000 # credit note, re-delivery, the customer call

straight_through = loaded / 60
error_rate = wrong / loaded

manual_minutes = EMAILS_PER_DAY * MINUTES_MANUAL
auto_minutes = EMAILS_PER_DAY * (1 - straight_through) * MINUTES_QUEUE
assisted_minutes = EMAILS_PER_DAY * MINUTES_REVIEW + EMAILS_PER_DAY * 0.12 * MINUTES_QUEUE

errors_per_day = EMAILS_PER_DAY * straight_through * error_rate
print(f"{'approach':<22}{'minutes/day':>13}{'labour Rs/day':>15}{'errors/day':>12}{'error cost Rs/day':>19}{'total':>10}")
for label, minutes, errors in [("manual (today)", manual_minutes, 0.0),
                              ("assisted: review each", assisted_minutes, 0.0),
                              ("straight through", auto_minutes, errors_per_day)]:
    labour = minutes / 60 * COST_PER_HOUR
    error_cost = errors * COST_OF_A_WRONG_ORDER
    print(f"{label:<22}{minutes:>13.0f}{labour:>15.0f}{errors:>12.1f}{error_cost:>19.0f}{labour + error_cost:>10.0f}")
```

approach	minutes/day	labour Rs/day	errors/day	error cost Rs/day	total
manual (today)	120	600	0.0	0	600
assisted: review each	40	198	0.0	0	198
straight through	9	47	6.7	13333	13380

Read the last column, not the second. Straight-through processing saves the most labour and costs the most money, because 19% of unattended orders are wrong and a wrong order costs far more than the two minutes it would have taken to check it.

The recommendation this chapter's own numbers force is the middle row: **assisted intake**. The model drafts every order, a person confirms it in about 45 seconds instead of typing it in three minutes, and nothing is written unseen. That saves around 90 minutes a day, introduces no silent errors, and keeps the coordinator in the loop who will notice when the model starts behaving oddly.

Straight-through processing becomes right when one of two things changes: the extraction gets good enough that the silent error rate falls below roughly 2%, or the cost of an error falls because something downstream catches it (a warehouse check, a customer confirmation email). **Both are worth engineering, and neither is a better prompt.**

Watch out: most automation business cases omit three costs. The exception queue (somebody works it, every day), the review and audit time (somebody samples the automatic ones), and the build and maintenance cost (this pipeline is a system that needs a version, tests, and an owner). Add those three and a lot of "80% cost reduction" slides become "30%", which is still excellent and is a number that survives contact with the finance team.

58.8 Rolling it out

Four stages, with a gate between each. Riverstone is at stage 2 and should stay there until the numbers say otherwise.

Stage	What runs	What a person does	Gate to the next stage
0. Shadow	the pipeline processes every email and writes nothing	works normally, unaware	the proposed orders match what the coordinator typed, on a week of real email
1. Assisted	the pipeline drafts every order	reviews and accepts each one	review takes under a minute and the correction rate is stable
2. Assisted with auto-load	small, clean orders load automatically	reviews the rest	the silent error rate on auto-loaded orders is measured and acceptable
3. Straight through	almost everything loads	samples, and works the queue	months of stable numbers, a downstream check, and an owner who watches

The gate between stages 2 and 3 is the one this chapter's data fails, and the failure is not a reason to stop: assisted intake already saves most of the labour. It is a reason to be specific about what would change the answer. Three things would:

- **Better extraction**, measured on the golden set, until the silent error rate is around 2%.
- **A downstream check** that catches a wrong order before it ships: a confirmation email to the customer listing the lines, which costs nothing and turns a silent error into a loud one.

- **A narrower automatic class:** only orders from the five customers who always send the same tidy table, where accuracy is near perfect. **Automate the strong segment fully rather than everything partially**, which is the single most useful tactic in this chapter.

```
import re

def style(text):
    if "|" in text:
        return "table"
    if re.search(r"^- ", text, re.MULTILINE):
        return "bullets"
    if "Forwarded message" in text:
        return "forwarded"
    if "would like to order" in text:
        return "prose"
    return "terse"

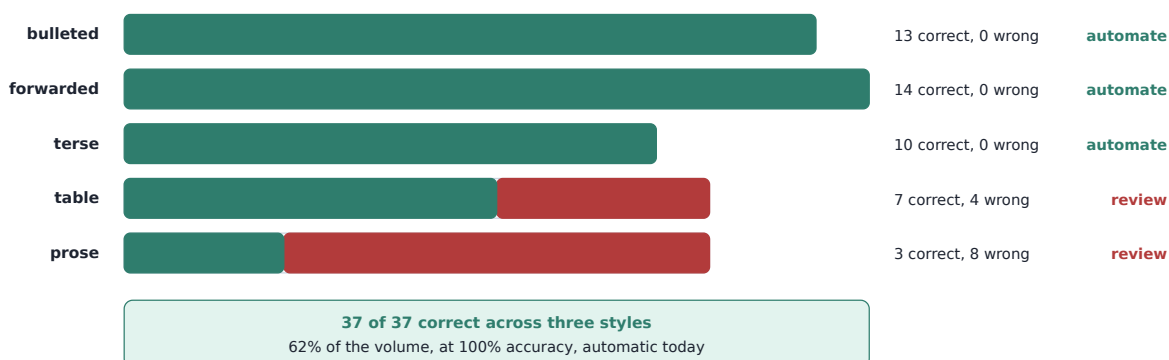
styles = {name: style(text) for name, text in emails.items()}
erp.rebuild()
run(emails, truth, approval_limit=10 ** 9)          # load everything, so every style can be scored
connection = erp.connect()

from collections import Counter
right, wrong_by_style = Counter(), Counter()
for row in connection.execute("SELECT order_id, source_email, po_number, customer, delivery_date
FROM orders"):
    want = truth[row["source_email"]]
    lines = [(line["product_code"], line["quantity"]) for line in
              connection.execute("SELECT product_code, quantity FROM order_lines WHERE order_id =
?",
                                (row["order_id"],))]
    matches = (row["po_number"] == want["po_number"] and row["customer"] == want["customer"]
               and row["delivery_date"] == want["delivery_date"]
               and sorted(lines) == sorted((i["product_code"], i["quantity"]) for i in
want["items"]))
    (right if matches else wrong_by_style)[styles[row["source_email"]]] += 1

print(f"{'email style':<12}{'loaded':>8}{'correct':>9}{'wrong':>7}  automate?")
for name in ("table", "bullets", "forwarded", "prose", "terse"):
    total = right[name] + wrong_by_style[name]
    verdict = "yes" if total and not wrong_by_style[name] else "no: review these"
    print(f"{'name':<12}{'total':>8}{'right[name]:>9}{'wrong_by_style[name]:>7}  {verdict}")
```

email style	loaded	correct	wrong	automate?
table	11	7	4	no: review these
bullets	13	13	0	yes
forwarded	14	14	0	yes
prose	11	3	8	no: review these
terse	10	10	0	yes

Automate the segment you are excellent at, not everything you are mediocre at



Nobody would have guessed this split: prose is hard, as expected, and tables are the second worst, because several items on one row are exactly what the extraction misses. You find the segment by measuring it, not by looking at the emails.

Figure 58.3 – 62% of the volume is already good enough to automate completely.

That is the case for segmenting, and it is not where anyone would have guessed. Bulleted, forwarded, and terse emails are extracted perfectly: 37 of 37 correct. The failures are concentrated in **prose** (3 of 11 right) and, surprisingly, in **tables** (7 of 11), where multiple items on one row are the exact weakness Chapter 54 measured.

So the automatic class is not "tidy emails" but "bulleted, forwarded, and terse", which together are **62% of the volume at 100% accuracy**, and the rest stays assisted. The general rule: **find the segment where you are already good enough, automate it completely, and leave the rest with a person**, rather than averaging your accuracy across both and being mediocre at everything. And note how you find that segment: by measuring, because the intuition that prose is hard and tables are easy is half wrong here.

58.9 The people part

Automating a task changes somebody's job, and pretending otherwise is both unkind and a good way to have your project quietly sabotaged.

What actually happens at Riverstone. The coordinator who typed forty orders a day now reviews forty drafts in forty minutes, works the exception queue, and spends the rest of the day on things that were always being squeezed out: chasing the orders that never arrived, calling customers whose emails are consistently unreadable, and checking that the ones the system passed actually shipped. Her job got more interesting and more responsible, and she is now the person who notices when the model starts behaving oddly, which is a role the system needs.

How to run the conversation:

- **Talk to her before you build**, not after. She knows which emails are awkward, which customers resend, and which "rules" finance wrote down but nobody follows. Every one of those is a requirement you would otherwise discover in production.

- **Be honest about the direction.** If the intent is eventually to need fewer people, say so, with a timeframe. People discover it anyway, and discovering it late poisons everything.
- **Make her the owner of the queue and the quality sample.** That is real authority over the system, not a consolation.
- **Measure her time, not her output.** "Orders processed" rewards rubber-stamping. "Corrections caught" and "queue cleared" reward the behavior you want.
- **Expect the first month to be slower.** Reviewing drafts while learning to trust them takes longer than typing. Say so in advance, or the pilot looks like a failure in week two.

Watch out: the person reviewing the queue is your best sensor, and the easiest one to break. Give them forty items with unhelpful reasons and they will start accepting everything within a week, and your quality metric becomes a fiction. A short queue with clear reasons is not a nicety; it is what keeps the measurement real.

58.10 When not to automate

Situation	Why not	Do instead
The process is broken	Automation makes bad outputs faster	Fix the process; it may then need no automation
Twice a month	Build cost never repays	A checklist, or a better template
Every case is judgment	There is no rule to encode	Give the person better information (Chapters 13, 16)
Silent, expensive errors	Exactly this chapter's finding	Assisted mode, or automate a narrow segment
The integration is the hard part	The model was never the problem	Fix the integration first; the model can wait
Nobody will own it	It decays, unobserved	Find an owner or don't build it
The input could be structured instead	A form beats extraction, every time	Ask the five biggest customers for a template or a portal

That last row deserves emphasis, because it is the cheapest win in the chapter and the one engineers dislike. **If two customers send 40% of your orders, a shared spreadsheet template removes the extraction problem entirely for 40% of the volume**, at the cost of one phone call each. Riverstone's real program is: templates for the top customers, assisted intake for everyone else, and straight-through for the segment where accuracy is proven. Nothing in that sentence is a model decision.

Common mistakes and how to spot them

Mistake	Symptom	Fix
No idempotency key	A resent email becomes two orders	A UNIQUE constraint on the source identifier, in the database
Application-level duplicate checks	Races create duplicates under load	Let the database enforce it
Writes without transactions	Orders with missing lines that look complete	One transaction per logical write
No audit log	"Who approved this?" has no answer	Source, action, actor, detail, timestamp, on every write
"The model proposed it" as accountability	Nobody is responsible	The pipeline version is the actor; a person's name when they approved
Quoting straight-through rate alone	88% sounds like success; 19% of it is wrong	Report the silent error rate beside it, always
ROI without the queue and review time	The business case disappoints in month two	Count every human minute the new process needs
ROI without the cost of being wrong	The most expensive option looks cheapest	Price an error and multiply
Automating everything at 80% accuracy	Mediocre across the board	Automate the segment where you are excellent
Guessing a foreign key	Orders against products that don't exist	Never invent a reference; reject to a person
A queue with unreadable reasons	Reviewers rubber-stamp within a week	Reasons in words, one screen, short queue
No batch-level circuit breaker	Sixty wrong orders instead of zero	Stop the run when the failure rate crosses a threshold
Surprising the person whose job changes	Quiet resistance, and requirements you never learn	Talk first, be honest, give them the queue
Extraction where a form would do	Solving a problem you could have removed	Templates and portals for the biggest senders

In the real world: the week the pilot was nearly cancelled

Riverstone's PO intake goes live in assisted mode in November. In week two, the sales head wants it switched off.

His complaint is specific: the coordinator is slower than she was. She processed forty orders a day by typing; now she reviews forty drafts and it is taking her longer, not less, because she is checking every field against the email.

Meera does two things. First, she measures rather than argues: review time is averaging 95 seconds in week one and 62 seconds in week two, against three minutes to type from scratch. It is already faster and still improving as trust builds. Second, she looks at *why* the reviews are slow, and finds that the

coordinator is re-checking the customer name and PO number on every order, because early in week one two of them were wrong.

That is the useful finding. Those two fields are **extracted with near-perfect accuracy**, measured on the golden set, and the two errors came from the same awkward forwarded-thread format. So the queue screen changes: the fields with proven accuracy are shown in grey as confirmed, the ones that vary are highlighted for checking, and forwarded threads are routed to the queue automatically. Review time drops to 41 seconds.

By January the numbers are: 40 emails a day, 41 seconds of review each, about 28 minutes of coordinator time against two hours before, no silent errors because nothing loads unseen, and an exception queue averaging five emails. The straight-through question comes back in March, and the answer is the segment version: the twelve customers who send tidy tables go automatic, everything else stays assisted.

What Meera tells the sales head: *"It was slower in week one because she was checking things we had already proven were right. We've made the screen show her what actually needs checking. It's now 28 minutes a day instead of two hours, and nothing reaches the ERP that a person hasn't seen."*

The lesson is that the interface is part of the automation. The model's accuracy was never the constraint in week two; the reviewer's uncertainty was, and that was fixed with layout and a routing rule.

Tools

Checked in September 2026:

- **Python 3.12** and **sqlite3** from the standard library: the whole order book, transactions and audit log in this chapter.
 - **Real systems of record:** PostgreSQL or the ERP's own API. The rules are identical: a uniqueness constraint, a transaction, an audit table.
 - **Workflow and orchestration:** Airflow, Dagster, Prefect, Temporal (Chapter 46). A daily intake run is a scheduled job with a queue in front of it.
 - **Queue and review interfaces:** a simple internal web page, a shared spreadsheet, or a workflow tool such as Camunda or a low-code form. **The interface matters more than the tool**, as the story above shows.
 - **RPA**, where there is no API: UiPath, Automation Anywhere, Power Automate. Treat as a bridge with an expiry date.
 - **Document intake** for photographed POs: a cloud document API, or a vision model (Chapter 54's multimodal section), feeding the same validation and queue.
 - **Companion files** in `ch58/`: `erp.py` (the SQLite order book with schema, idempotency, transactions, and audit log), `intake.py` (the pipeline), and `ch58_check.py`.
-

The project: the intake pipeline, end to end

Goal: an automation that writes to a system of record safely, and a business case that would survive a finance review.

Steps:

1. **Build the target table** with a uniqueness constraint on your source identifier and an audit table beside it.
2. **Assemble the pipeline:** read, extract, parse tolerantly, validate, decide, write in a transaction, log.
3. **Replay your whole input twice** and prove nothing is written twice.
4. **Measure both numbers:** straight-through rate and silent error rate, against ground truth.
5. **Sweep the approval threshold** and record what each level costs in review time and permits in wrong value.
6. **Price it:** manual, assisted, and straight through, including queue time, review time, and the cost of an error.
7. **Design the queue screen:** email beside proposal, reasons in words, accept/correct/reject.
8. **Find your good segment** and quantify how much of the volume it covers.
9. **Write the rollout plan** with the gate for each stage.
10. **Write the runbook:** what happens when the provider is down, when the ERP rejects a write, and when the failure rate crosses the batch threshold.

Deliverables: the pipeline, the replay proof, the two rates, the cost table, the queue design, the segment analysis, and the rollout plan.

Stretch goals:

- Add a confirmation email to the customer listing the lines, and argue how that changes the straight-through gate.
- Add a second source (a photographed PO) routed through the same validation and queue.
- Add the batch circuit breaker and demonstrate it stopping a run.
- Build the reviewer screen and time yourself reviewing twenty orders.
- Model the ROI at 4× the volume, and find where the queue becomes a full-time job.

You've got it when...

- ☐ I can tell RPA, integration, and model-assisted automation apart and pick the right one.
- ☐ I rank candidates by volume × time, then filter by the cost of an error.
- ☐ Every automated write has a uniqueness key, a transaction, and an audit row.
- ☐ I can replay an entire day's input and prove nothing doubles.
- ☐ I report the silent error rate beside the straight-through rate.
- ☐ My approval threshold is chosen from money, not from habit.

- ☐ I classify exceptions, and each class has a defined action.
 - ☐ My ROI arithmetic includes the queue, the review, the build, and the cost of being wrong.
 - ☐ I roll out in stages with a gate at each, and I know which gate I am at.
 - ☐ I automate the segment I am excellent at rather than everything I am mediocre at.
 - ☐ I talk to the person whose job changes before I build.
-

Recap

- **Three generations:** RPA (a bridge), integration (the durable answer), and model-assisted (for unstructured input). Riverstone needs the third feeding the second.
 - **Choose by volume, variability, rules, and the cost of an error.** Purchase orders score well on three and badly on the fourth.
 - **Writing is different from reading.** Every automated write needs an **idempotency key in the database**, a **transaction**, and an **audit log** naming the pipeline version as the actor.
 - **Two gates:** validation holds malformed output, and an **approval threshold** holds large output. Both are business decisions with measurable costs.
 - **Classify exceptions**, and give the batch a circuit breaker: sixty wrong orders is worse than none.
 - **Report both rates.** Riverstone's pipeline is **88% straight through and 19% silently wrong**, which are the same system described two ways.
 - **Honest ROI** counts the queue, the review, the build, and the errors. On Riverstone's numbers, manual costs ₹600 a day, **assisted ₹198**, and straight through **₹13,380**, because unattended errors dominate everything.
 - **Roll out in stages:** shadow, assisted, assisted with auto-load, straight through, with a measured gate at each.
 - **Automate the segment you are excellent at.** The table-style emails were extracted perfectly and could go automatic today.
 - **The interface is part of the automation**, and the person reviewing the queue is your best sensor. Do not break them with a long queue and bad reasons.
-

Practice exercises

Work in `companion/ch58`.

Warm-up

1. Rebuild the ERP, run the intake, and query the three tables. How many rows are in each, and why is the audit table the largest?
2. Run the intake twice without rebuilding. What changes in `orders`, and what changes in `audit_log`?

3. Find the largest order the pipeline loaded automatically, and the largest it held for approval. Is the boundary where you expected?
4. Which validation reasons appear in the held queue, and how many of each?

Core

5. Set the approval limit to ₹25,000 and to ₹500,000. Report straight-through rate, approvals, and the wrong value that would have been auto-loaded at each.
6. Break the uniqueness constraint (remove `UNIQUE` from `source_email`), rerun the replay, and show what happens. Then put it back.
7. Force a failure halfway through writing an order's lines, and prove the transaction rolls back the header too.
8. Compute the silent error rate separately for each email style (table, prose, forwarded, bullets, terse). Which segment would you automate?
9. Redo the ROI table with a cost of error of ₹500 and of ₹8,000. At what cost of error does straight-through processing become the cheapest option?
10. Add a rule that any order from a customer with more than one PO in the same email goes to the queue. How many does it catch?

Stretch

11. Add a `confirmations` table and generate a customer confirmation email listing the lines. Argue, with numbers, how it changes the straight-through gate.
12. Implement the batch circuit breaker: stop the run if more than 20% of emails fail validation, and log why.
13. Add an `approvals` table recording who approved each pending order and when, and produce the audit query a compliance officer would ask for.
14. Write the reviewer screen as a small HTML page showing the email beside the proposed order with the uncertain fields highlighted.

Think about it (no code needed)

15. Your pipeline is 95% straight through and 3% silently wrong. Would you ship it? What would you need to know first?
16. The ERP team offers you an API that is slower but validates everything server-side. Do you take it?
17. Finance asks why the coordinator is still needed if the system is 88% automatic. What do you say?
18. Which parts of this chapter would change if the automation sent *money* rather than created an order?

Key terms

intelligent automation · RPA · screen scraping · API integration · model-assisted intake · straight-through processing · silent error rate · system of record · idempotency · idempotency key · unique constraint · transaction · rollback · audit log · actor · pipeline version · human in the loop · assisted mode · approval threshold · exception queue · correction rate · exception taxonomy · transient failure · malformed input · unknown reference · batch circuit breaker · touch time · ROI · cost of an error · segmentation · shadow rollout · gate · reviewer interface · confirmation loop · process redesign · structured input (templates and portals)

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 59, Industry Case Studies**, puts this pipeline beside eight other end-to-end projects.
 - **Chapter 57, LLMOps**, keeps the extraction honest once this is running daily.
 - **Chapter 46, Pipelines & Orchestration**, schedules the intake run and the queue reminders.
 - **Chapter 12 and 28, SQL**, are where the transaction, the constraint, and the audit table come from.
 - **Chapter 64, Responsible AI & Governance**, covers accountability for an automated decision and what customers must be told.
 - **Chapter 3, How a Business Runs on Data**, is where the order-to-cash process this automates was first explained; reread it with this chapter in mind.
 - **Chapter 74, Machine Learning & AI Question Bank**, has the interview version: "how would you automate invoice processing, and how would you know it was safe?"
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1.

```
import sys
sys.path.insert(0, ".")
import erp
from intake import run
from pipeline import load_emails

erp.rebuild()
truth, emails = load_emails()
run(emails, truth)
connection = erp.connect()
for table in ("orders", "order_lines", "audit_log"):
    count = connection.execute(f"SELECT COUNT(*) AS n FROM {table}").fetchone()["n"]
    print(f"{table:<12}{count:>5}")
```

orders	59
order_lines	97
audit_log	119

The audit table is the largest because **every email produces at least one audit row whether or not it produces an order**: a `proposed` entry for all sixty, plus a `loaded` or `awaiting_approval` entry for those written. That ratio is healthy. An audit log the same size as the orders table means you are only recording successes, which is exactly the half you will not need during an incident.

2. `orders` and `order_lines` do not change at all; `audit_log` grows by a `proposed` and a `duplicate_ignored` row for every email. That is the behavior to test for in CI: **run the pipeline twice in the test and assert the order count is unchanged**, because idempotency is the property most likely to be broken by a well-meaning refactor.

3.

```
connection = erp.connect()
for status in ("loaded", "awaiting_approval"):
    row = connection.execute("SELECT po_number, customer, order_value FROM orders WHERE status = ?
    "
                              "ORDER BY order_value DESC LIMIT 1", (status,)).fetchone()
    if row:
        print(f"largest {status:<18} {row['po_number']} {row['customer']:<22} Rs
{row['order_value']:>10,.0f}")
```

largest loaded	P0-92544 Green Leaf Hotels	Rs	89,150
largest awaiting_approval	P0-95195 Sharma Hardware	Rs	145,800

The largest automatic order sits just under the ₹100,000 limit and the smallest held one just above, which is what a threshold does. The question to ask next is whether ₹100,000 is where the risk changes, and it usually isn't: risk tracks the customer, the product, and how unusual the order is, which is the argument for a richer rule (exercise 10) once the simple one is working.

4.

```
from collections import Counter
erp.rebuild()
result = run(emails, truth)
reasons = Counter(reason for item in result["queue"] for reason in item["reasons"])
print("reasons items reached the queue:")
for reason, count in reasons.most_common():
    print(f" {count:>2} {reason}")
```

```
reasons items reached the queue:
1 order value Rs 145,800 is above the Rs 100,000 approval limit
1 order value Rs 121,250 is above the Rs 100,000 approval limit
1 order value Rs 119,500 is above the Rs 100,000 approval limit
1 order value Rs 115,000 is above the Rs 100,000 approval limit
1 order value Rs 100,500 is above the Rs 100,000 approval limit
1 order value Rs 105,000 is above the Rs 100,000 approval limit
1 no order lines
```

Most items in this queue are there for **value**, not for a defect, which is a healthy queue. A queue dominated by one validation reason is a signal to fix that reason in the pipeline rather than to hire another reviewer: the correction rate per reason (section 58.5) is the metric that tells you which one to fix first.

5.

```
def wrong_value(limit):
    erp.rebuild()
    outcome = run(emails, truth, approval_limit=limit)["counts"]
    local = erp.connect()
    bad = 0.0
    for row in local.execute("SELECT order_id, source_email, order_value, po_number, customer,
delivery_date "
                            "FROM orders WHERE status = 'loaded'"):
        want = truth[row["source_email"]]
        lines = [(line["product_code"], line["quantity"]) for line in
                  local.execute("SELECT product_code, quantity FROM order_lines WHERE order_id = ?",
                                (row["order_id"],))]
        if not (row["po_number"] == want["po_number"] and row["customer"] == want["customer"]
                and row["delivery_date"] == want["delivery_date"]
                and sorted(lines) == sorted((i["product_code"], i["quantity"]) for i in
want["items"]))):
            bad += row["order_value"]
    return outcome.get("loaded", 0), outcome.get("awaiting_approval", 0), bad

for limit in (25_000, 100_000, 500_000):
    loaded, approvals, bad = wrong_value(limit)
    print(f"limit Rs {limit:>7,:} loaded {loaded:>2}, approvals {approvals:>2}, "
          f"wrong value auto-loaded Rs {bad:>10,.0f}")
```

limit Rs 25,000:	loaded 16,	approvals 43,	wrong value auto-loaded Rs	57,950
limit Rs 100,000:	loaded 53,	approvals 6,	wrong value auto-loaded Rs	363,550
limit Rs 500,000:	loaded 59,	approvals 0,	wrong value auto-loaded Rs	579,050

The wrong value auto-loaded roughly doubles with each step up. **That is the number to take to the business**, because "how much incorrectly-recorded order value are we willing to create per week?" is a question a finance director can answer and "what confidence threshold should we use?" is not.

6. Without the constraint, the replay writes a second copy of every order, and the audit log shows `loaded` twice for the same email. The `duplicate_ignored` branch never fires, because it depends on finding the existing row, and under concurrency even that check loses races. The exercise is worth doing once, because seeing 118 orders where 59 belong is more persuasive than any explanation of why the constraint belongs in the database rather than in the application.

7. Raise an exception inside the loop that writes lines (a bad product code, or a deliberate `raise`). The `with connection:` block rolls back, so the header disappears with the lines and the email stays unprocessed, ready to retry. Without the transaction you would have an order with a header and two of its three lines, which is the failure that ships short and is discovered by a customer. **Test this deliberately**, because it is the one failure mode that never happens during development and always happens eventually.

8. Section 58.8 does this: bulleted, forwarded, and terse emails are 37 of 37 correct; prose is 3 of 11 and tables 7 of 11. Automate the first three, which is 62% of the volume, and keep the rest assisted. The reason tables fail is Chapter 54's: several items on one row, where the extraction takes the first and misses the rest. **That is also a fixable engineering task**, so the segment that is automatic should grow over time and the analysis should be rerun each quarter.

9. At ₹500 an error, straight-through costs about ₹3,400 a day against assisted's ₹198, so assisted still wins comfortably. At ₹8,000 an error the gap becomes absurd. The break-even is roughly where $\text{error rate} \times \text{cost of error} \times \text{volume}$ falls below the labour saved, which at Riverstone's 19% silent error rate means an error would have to cost under about ₹15 for straight-through to win. **The lever is the error rate, not the cost**: at a 2% silent error rate the arithmetic reverses, which is why section 58.8's gate is written in those terms.

10. Count `PO-` matches in the email text and route anything with more than one to the queue. It catches the forwarded threads containing an earlier order, which are exactly the emails where the extraction silently takes the wrong PO number. **A cheap textual rule that routes a known-hard case to a person is worth more than a prompt change**, because it is deterministic, testable, and it cannot regress when the model changes.

11. The confirmation turns a silent error into a loud one: the customer reads the lines and replies when they are wrong, usually within a day and always before dispatch. That changes the straight-through gate fundamentally, because the cost of an error drops from a wrong delivery (₹2,000-ish, plus goodwill) to an email exchange. Rerun exercise 9's arithmetic with an error cost of about ₹200 and straight-through becomes defensible even at this accuracy. **The cheapest way to make automation safe is often to add a check outside your system**, and this one also improves the customer's experience.

12. Count validation failures as you go; if more than 20% of the batch has failed by the halfway point, stop, log `batch_aborted` with the counts, alert, and process nothing further. The subtlety worth getting right: **abort rather than pause**, and make the run resumable, so that when the cause is fixed the same emails are reprocessed from the start with idempotency protecting anything already written.

13. The compliance query is the one to design for: "for order 4471, show me what was proposed, by which pipeline version, what was written, who approved it, and when." A single join across `audit_log` and `approvals`, keyed on the source identifier, answers it. Two design points: store the approver's identity from your authentication system rather than a typed name, and never allow an approval row to be deleted, only superseded.

14. The layout is the lesson, not the code: the email on the left, the proposed order on the right, fields with proven accuracy shown in grey as confirmed, uncertain fields highlighted, and three buttons (accept, correct, reject). The real-world story in this chapter is entirely about this screen: review time fell from 95 seconds to 41 because the reviewer stopped checking fields that were already known to be right.

15. Probably yes, with three things first. **What is the cost of one of those errors?** 3% of a high-value flow can be worse than 19% of a low-value one. **Is there a downstream check** that catches errors before they matter? **Who samples the automatic ones, how often, and what happens when the rate**

moves? If all three have answers, ship it to the segment where it is strongest and keep the rest assisted. If nobody will own the sampling, do not ship it: an unmonitored 3% becomes an unknown 10% within a year.

16. Take it, almost always. Server-side validation means the ERP rejects what it cannot accept, which converts a class of silent errors into loud ones at the moment of writing, and it is a contract you do not have to maintain. Latency rarely matters for a batch intake, and if it does, parallelize. The only reason to decline is if "slower" means minutes per record at a volume that makes the run impossible, and even then the right conversation is with the ERP team, not a workaround.

17. *"Because 12% of emails need judgment, and someone has to give it. She works the queue, she approves the large orders, and she samples the automatic ones, which is how we know the system still works. Her time on the task has gone from two hours a day to about half an hour, and that half hour is the part that protects us from shipping the wrong thing."* If the real question is whether headcount can fall, answer it directly with the measured time saved and let the business decide, rather than letting the question live unasked.

18. Almost everything gets stricter. **Approval thresholds fall to near zero**, because the cost of an error is immediate and often unrecoverable. **Two-person authorization** becomes standard above small amounts. **Idempotency stops being a nicety** and becomes the property that prevents paying twice, and it must survive crashes, not just retries. The **audit trail becomes a legal record** with retention requirements. Reconciliation against the bank becomes a daily control, not a monthly one. And **you almost certainly do not let a model decide the amount**: it can read the invoice, and a rule, a match against a purchase order, and a person decide whether to pay it. The general principle: as the cost and irreversibility of an action rise, the model's role shrinks to reading, and the decision moves to rules and people.